Topic 9: Closures and Higher-Order Functions

Task 1: Guided Activity, Student result analysis

```
struct Student {
    let name: String
    let marks: Int
    let dept: String
}
let students = [
    Student(name: "Asha", marks: 82, dept: "IT"),
    Student(name: "Ravi", marks: 35, dept: "CSE"),
    Student(name: "Meena", marks: 91, dept: "IT"),
    Student(name: "Kiran", marks: 56, dept: "ECE"),
    Student(name: "Divya", marks: 67, dept: "CSE"),
    Student(name: "Arjun", marks: 40, dept: "IT")
]
let passedStudents = students.filter { student in
    student.marks >= 40
}
print("Passed Students:")
print(passedStudents.map { $0.name })
let names = students.map { $0.name }
print("Student Names:", names)
let totalMarks = students.map { $0.marks }.reduce(0, +)
let average = Double(totalMarks) / Double(students.count)
print("Class Average:", average)
let rankedStudents = students.sorted { $0.marks > $1.marks }
print("Rankings:")
rankedStudents.forEach {
    print("\($0.name): \($0.marks)")
}
let groupedStudents = Dictionary(grouping: students, by: { $0.dept })
```

```
print("Students by Department:")
for (department, list) in groupedStudents {
    print(department, list.map { $0.name })
}
let result1 = students.filter { student in
    student.marks >= 40
}
let result2 = students.filter {
    $0.marks >= 40
}
print(result1.map { $0.name })
print(result2.map { $0.name })
```

Output:

```
Passed Students:
["Asha", "Meena", "Kiran", "Divya", "Arjun"]
Student Names: ["Asha", "Ravi", "Meena", "Kiran", "Divya", "Arjun"]
Class Average: 61.833333333333336
Rankings:
Meena: 91
Asha: 82
Divya: 67
Kiran: 56
Arjun: 40
Ravi: 35
Students by Department:
CSE ["Ravi", "Divya"]
ECE ["Kiran"]
IT ["Asha", "Meena", "Arjun"]
["Asha", "Meena", "Kiran", "Divya", "Arjun"]
["Asha", "Meena", "Kiran", "Divya", "Arjun"]
```

Task 2: Practice Task, Write your own higher-order functions

```
func myFilter(_ items: [Int], _ predicate: (Int) -> Bool) -> [Int] {
    var result: [Int] = []
    for item in items {
        if predicate(item) {
            result.append(item)
        }
    }
    return result
}
let numbers = [1, 2, 3, 4, 5, 6]
let evenNumbers = myFilter(numbers) { number in
    number % 2 == 0
}
print("Even numbers:", evenNumbers)
func applyTwice(_ value: Int, _ f: (Int) -> Int) -> Int {
    return f(f(value))
}
let answer = applyTwice(5) { number in
    number * 2
}
print("Apply twice:", answer)
class Button {
    var handlers: [() -> Void] = []
    func onTap(_ handler: @escaping () -> Void) {
        handlers.append(handler)
    }
    func tap() {
        for handler in handlers {
            handler()
        }
```

```
    }
}
let button = Button()
button.onTap {
    print("First handler called")
}
button.onTap {
    print("Second handler called")
}
button.tap()
```

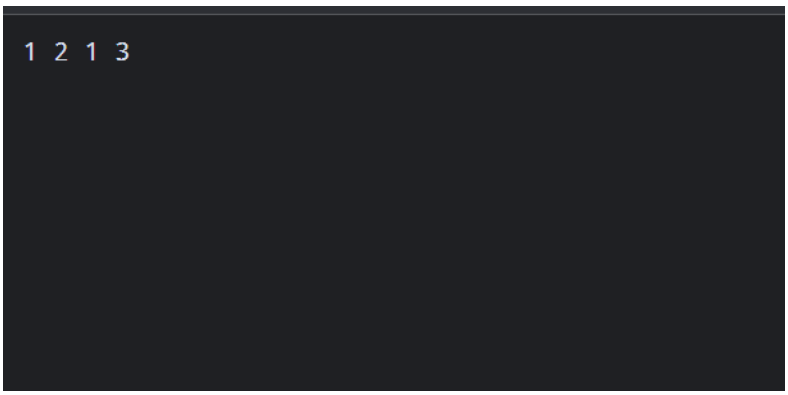
Output:

```
Even numbers: [2, 4, 6]
Apply twice: 20
First handler called
Second handler called
```

Task 3: Predict Task, Closure capture

```
func makeCounter() -> () -> Int {
    var count = 0
    return {
        count += 1
        return count
    }
}
let a = makeCounter()
let b = makeCounter()
print(a(), a(), b(), a())
```

Output:



Explanation :

- a = makeCounter() creates a counter with its own count = 0.
- b = makeCounter() creates another independent counter with its own count = 0.
- Each closure remembers its own count variable. This is called closure capture.

The print statement executes in this order:

1. a() → 1
2. a() → 2
3. b() → 1
4. a() → 3

Why don't a and b interfere?

Each call to makeCounter() creates a separate local variable. Therefore, changing a's count does not change b's count. This is because each closure captures its own variable.